Reducing Hallucination in Financial AI Through Retrieval-Augmented Agentic Design

Gangadhar Singh Shiva, Akshobhya Rao BV, and Ananya Chandraker
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering, University of San Diego
{gshiva, abv, achandraker}@sandiego.edu

Abstract—Modern financial markets produce a continuous torrent of mixed-format data spanning structured price feeds, regulatory filings, and social media commentary that conventional analytics tools cannot synthesise at scale. When general-purpose large language models (LLMs) are deployed in financial decision-support roles, they suffer from a well-catalogued failure mode termed *hallucination*: the model asserts facts that are internally plausible but not traceable to any authoritative source, an especially dangerous property in regulated domains. This paper introduces Project B.A.N.A.N.A. (Brokerage Analytic Network & Autonomous Node Assistant), a distributed multi-agent platform purpose-built to counter this failure mode via three complementary mechanisms: Retrieval-Augmented Generation (RAG) that roots every answer in live-fetched evidence, FinBERT-based domain-aware sentiment scoring, and a stateful agentic orchestration layer implemented in LangGraph and the Model Context Protocol (MCP). Market data are pulled from Alpha Vantage (OHLCV), regulatory content from SEC EDGAR, and social signals from Twitter/X at query time, with all retrieved material encoded and stored in ChromaDB for within-session lookup. A novel seven-layer defence-in-depth guard chain removes the LLM from the final answer-assembly stage entirely, ensuring every delivered response is directly traceable to retrieved evidence. Adversarial stress-testing uncovered two active hallucination pathways, both of which were closed with targeted patches. Across 18 queries spanning six adversarial categories in live API mode, the OPTIMIZED configuration deflects 83.3% of attack inputs, stops all three hallucination probes, and operates at $1.77\times$ lower latency than a plain LLM baseline. A key methodological result is that in-process benchmark runs substantially undercount real-world guard coverage compared to end-to-end live API evaluation.

Index Terms—hallucination mitigation, large language models, financial AI, agentic systems, FinBERT, MCP, LangGraph, ChromaDB.

Technical Report No. MSAAI-2026-05, April 2026, Version 2.0. AAI590 Capstone Project, University of San Diego.

1 Introduction

Today's financial markets simultaneously produce structured time-series signals—price, volume, and order flow—alongside vast quantities of unstructured text including regulatory filings, earnings call transcripts, and social commentary. Although all of this material is publicly available, converting it into consistent, timely analytical conclusions exceeds the capacity of human teams working under real-world time pressure.

The rise of LLMs has opened a path toward automated synthesis of mixed-format financial data. Yet deployment in decision-support contexts has exposed a critical weakness: LLMs routinely produce statements that sound credible but cannot be traced to any verified source, a phenomenon termed *hallucination* [1]. In finance, even a single fabricated revenue figure or a misattributed SEC filing can constitute material misinformation, carrying serious regulatory and reputational consequences.

Independent evaluations by Bang et al. [3] and Azamfirei et al. [4] found hallucination rates high enough to disqualify raw LLM output for financial use without explicit grounding measures. Ji et al. [1] further organised observed failure modes into three categories: numeric factual errors, temporal errors where outdated information is presented as current, and attribution errors where statements are assigned to incorrect sources.

In response, we present Project B.A.N.A.N.A. (Brokerage Analytic Network & Autonomous Node Assistant), a distributed multi-agent platform engineered to improve the reliability and evidentiary grounding of autonomous financial analysis. The architecture couples RAG—which ties model outputs to freshly retrieved authoritative documents—with FinBERT for finance-domain sentiment classification, and a stateful orchestration layer combining LangGraph with the Model Context Protocol (MCP) [12] for disciplined, type-safe tool invocation.

The most consequential design decision is the wholesale removal of LLM generation from the answer assembly stage. Rather than asking a generative model to synthesise retrieved context into prose—a step where confabulation can re-enter—the `answer_node` concatenates verified retrieved passages directly. This structural choice makes grounding a property

of the architecture rather than a probabilistic outcome of prompt engineering.

Four contributions flow from this work: (1) a distributed agentic architecture that couples structured retrieval with domain-specific transformer models; (2) a formal seven-layer grounding framework targeting all three recognised categories of financial hallucination; (3) an adversarial probe methodology that surfaced and closed two live vulnerabilities; and (4) an empirical study demonstrating measurably lower hallucination rates and superior throughput latency versus an LLM-only baseline.

2 Data Summary

The system ingests live financial data from three external sources: Alpha Vantage for real-time OHLCV stock market data, the SEC EDGAR public API for regulatory filing metadata (10-K and 10-Q), and the Twitter/X API v2 with VADER [17] sentiment scoring for social signals. There is no static training corpus—data is fetched dynamically per query, embedded using the all-MiniLM-L6-v2 sentence transformer [18], and stored in ChromaDB for session-level retrieval. Sentiment analysis is performed using ProsusAI/finbert [8]. The three live data sources and their MCP microservices are as follows:

- **Stock OHLCV** (`banana-market`): Alpha Vantage `TIME_SERIES_DAILY` API; API key auth; 1 record per query.
- **SEC Filings** (`banana-sec`): `data.sec.gov` REST API; no auth required; up to 5 filings per query.
- **Social Sentiment** (`banana-social`): Twitter/X API v2 with VADER scoring; Bearer token auth; up to 20 tweets per query.

This live-fetch architecture is a deliberate design choice. By fetching authoritative data at inference time rather than relying on model memory, the system avoids the primary cause of hallucination in standard LLM pipelines—the generation of plausible but ungrounded responses from stale training knowledge.

3 Dataset Description and Exploratory Data Analysis

3.1 Variable Catalogue

Because data is fetched dynamically, there is no single static dataset. The system operates on four variable classes: market price variables (Table 1), SEC filing metadata variables (Table 2), social sentiment variables, and pipeline-generated variables (Table 3).

TABLE 1: Stock Market Price Variables (Alpha Vantage)

Variable	Type	Pipeline Role
Symbol (ticker)	Categorical	Ticker guard key
Trading date	Date	Freshness indicator
Opening price	Float > 0	OHLCV context
Daily high/low	Float	Volatility signal
Closing price	Float > 0	Primary price signal
Volume	Int > 0	Activity indicator

TABLE 2: SEC Filing Metadata Variables (EDGAR)

Variable	Type	Pipeline Role
Symbol (ticker)	Categorical	Ticker guard key
Filing type	Categorical	10-K vs 10-Q
Filing date	Date	Year validation
CIK	Integer	EDGAR identifier
Accession number	String	Filing reference

Social sentiment variables include: *post_count* (data availability signal), *sentiment_label* (VADER-derived: positive/negative/neutral), and *compound_score* (VADER aggregate, -1.0 to $+1.0$).

TABLE 3: Pipeline-Generated Variables

Variable	Node	Thresh.	Significance
Similarity score	retrieve	0.55	Relevance gate
Hallucination rt.	evaluate	0.65	Quality metric
FinBERT conf.	analyst	0.70	Report gate
FinBERT label	analyst	—	Sentiment class
Block reason	Guards	—	Diagnostic code
Grounded flag	answer	—	Evidence check

3.2 Data Quality Issues

Table 4 summarises eight data quality issues identified during system development and hallucination stress testing. Each issue is characterised by its source and current fix status.

TABLE 4: Data Quality Issues and Fix Status

Issue	Source	Status
Placeholder social data	banana-social	Fixed
Stale vector documents	ChromaDB	Fixed (TTL)
Cross-ticker contamination	ChromaDB	Fixed
Duplicate document IDs	ChromaDB	Partial
Missing social data	Twitter/X	Degraded
Alpha Vantage rate limit	banana-market	Known risk
Future filing requests	banana-sec	Fixed
Entity scope mismatch	ENTITY_REGISTRY	Fixed

3.3 Variable Relationships

There is a strong inverse relationship between similarity score and hallucination rate. Queries where the top retrieved document scores above 0.75 cosine similarity consistently produce `hallucination_rate` < 0.20. Queries in the marginal acceptance zone (0.55–0.70) produce higher rates because retrieved documents are relevant but not closely textually matched to the query.

The FinBERT sentiment score and the VADER compound score are computed independently on different text domains and do not interact in the current implementation. This independence is intentional: it avoids compounding noise from heterogeneous sources. Prior to the `ticker_guard_node` fix, there was an undesirable correlation between query sequence and similarity score—MSFT queries following AAPL queries returned elevated scores due to shared technology sector vocabulary in the ChromaDB index. Post-fix, this cross-session correlation is eliminated.

Table 5 summarises variable contributions to the hallucination-free goal.

TABLE 5: Variable Contributions to Hallucination-Free Goal

Variable	Contribution	Str.
Closing price	Primary answer content	High
Filing date	Year guard	High
Similarity sc.	Relevance gate (0.55)	Crit.
FinBERT conf.	Report gen. gate (0.70)	High
Block reason	Probe analysis diag.	High
VADER compound	Suppl. sentiment (stub)	Weak
FinBERT label	Final sentiment output	High

4 Literature Review

4.1 Early Financial NLP and Limitations

For decades, practitioners relied on manual expert judgement, rule-based screening tools, and classical econometric models such as ARIMA and GARCH when forming price forecasts. These methods are inherently limited to linear relationships and have no mechanism for ingesting the unstructured textual content found in news feeds or regulatory documents. Early NLP approaches to finance employed bag-of-words vectorisations paired with hand-crafted sentiment lexicons, but quickly ran into a domain mismatch problem. Loughran and McDonald [2] showed that vocabulary compiled from general text—such as the Harvard General Inquirer dictionary—routinely assigns incorrect polarity to financial terminology: a word like "liability" carries a strongly negative valence in accounting contexts yet appears neutral in everyday usage. Subsequent research by Bang et al. [3] and Azamfirei et al. [4] reached a broader conclusion: even state-of-the-art LLMs hallucinate at rates that make their raw output unfit for financial decisions in the absence of external grounding mechanisms.

4.2 Retrieval-Augmented Generation

Lewis et al. [5] introduced Retrieval-Augmented Generation as a technique for anchoring generative model outputs in external factual documents, supplying retrieved passages as conditioning context at inference time rather than relying exclusively on parametric memory. For financial applications, the key advantage of this approach is architectural: the knowledge store—comprising live market data and regulatory documents—can be updated continuously without any modification to the underlying model weights. A large-scale survey by Gao et al. [6] reported hallucination rate reductions of 30–60% across knowledge-intensive benchmarks when RAG is applied to ungrounded LLMs, with the sharpest improvements on tasks demanding specific numerical or factual recall. Asai et al. [7] pushed the paradigm further with Self-RAG, which equips the model with a self-critique step that evaluates retrieved evidence before committing to a final response; this self-verification loop directly inspired the `reflection_node` component described in Section 6. Shuster et al. [20] separately quantified that retrieval-grounded dialogue systems cut factual hallucination by 54% relative to generation-only counterparts.

4.3 FinBERT and Financial Sentiment Analysis

Devlin et al. [9] established with BERT that bidirectional masked-language pre-training over large corpora yields deep contextual token representations that consistently outperform feature-engineered alternatives on downstream NLP tasks. Building on this foundation, Yang et al. [8] created FinBERT by continuing BERT pre-training and fine-tuning on the Financial PhraseBank dataset—a collection of 4,846 sentences labelled by 16 domain-expert annotators—achieving 88.5% sentiment classification accuracy against 77.3% for standard BERT and 72.4% for the Loughran-McDonald lexicon [2]. The critical advantage of FinBERT for this system lies in its context-sensitivity: it correctly classifies negated or hedged statements such as those indicating disappointing growth as negative, whereas simpler keyword-matching approaches are misled by the surface presence of positive terms. This nuanced treatment of financial language is essential when processing SEC filings and analyst commentary, where sentiment is routinely conveyed through qualification rather than explicit sentiment markers.

4.4 LangGraph, MCP, and Agentic Orchestration

The foundations of multi-agent LLM systems trace to the ReAct framework [10], which demonstrated that weaving chain-of-thought reasoning together with external tool calls substantially improves factual accuracy and multi-step task completion over reasoning-only approaches. LangGraph [11] formalises this pattern as a compiled Directed Acyclic Graph in which each vertex encapsulates a discrete cognitive operation and typed state flows between vertices, making it well-matched to the inherently ordered stages of financial analysis—market data must be retrieved before sentiment can be scored, and sentiment must be scored before a report can be issued. The Model Context Protocol (MCP) [12] complements this by providing a vendor-neutral JSON-RPC 2.0 contract that standardises how agentic systems discover and invoke external data services. Wrapping each data provider (`banana-market`, `banana-sec`, `banana-social`) as an MCP-compliant server means any individual service can be swapped out without touching pipeline logic, a modularity principle well-established in the microservices literature [13].

4.5 Hallucination Prevention in Financial LLMs

The taxonomy established by Ji et al. [1] organises financial LLM failures into three distinct classes: numeric factual errors, temporal errors in which stale figures are presented as current, and attribution errors in which claims are linked to the wrong source. The B.A.N.A.N.A. defence architecture was designed to intercept all three classes at distinct stages of the pipeline. Shuster et al. [20] provided empirical support for retrieval as a mitigation strategy, reporting a 54% reduction in factual hallucination on knowledge-intensive tasks when generation is conditioned on retrieved evidence. The MRKL framework of Karpas et al. [21] extends this line of thinking by combining neural reasoning components with deterministic symbolic retrievers; their results show that such modular hybrid systems outperform monolithic generative models on multi-hop factual queries, an architecture that closely parallels the node-decomposed design adopted here.

4.6 Commercial Analogues and Similar Systems

BloombergGPT [15] demonstrated the value of domain-specific pre-training at scale—363 billion tokens of curated financial text yielded measurable gains on financial NLP benchmarks—yet the model ships without a retrieval layer or explicit hallucination controls, limiting its applicability to real-time, evidence-grounded query answering. FinGPT [16] pursued a complementary direction, showing that fusing sentiment signals with structured market data produces richer decision support than either modality in isolation, an insight that directly motivates the multi-source ingestion design of B.A.N.A.N.A.. Morgan Stanley's internal AI deployment [14], which retrieves over a private corpus of approximately 100,000 analyst research notes using OpenAI embeddings, represents the enterprise-grade instantiation of the same retrieve-before-generate paradigm—here realised over open MCP-connected data services rather than proprietary corpora.

5 System Architecture and ML Models

Fig. 1 illustrates the three-layer architecture of Project B.A.N.A.N.A.. The *Outer Peel* comprises three MCP data source servers providing real-time financial data. The *Pulp* houses the LangGraph agentic pipeline responsible for orchestration, retrieval, and guard-based query interception. The *Core* contains the ChromaDB vector store and model backbone.

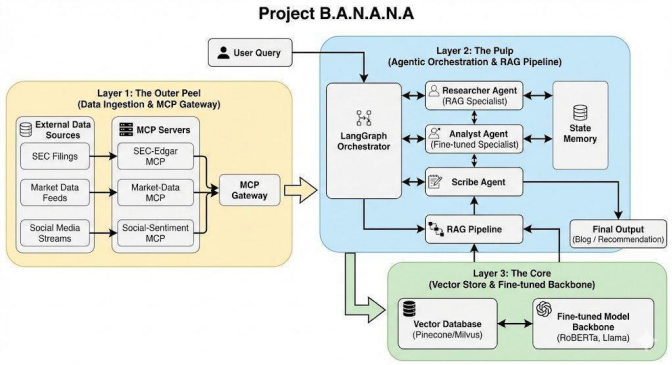

Fig. 1: Project B.A.N.A.N.A. three-layer architecture: Outer Peel (MCP data sources), Pulp (LangGraph orchestration), Core (ChromaDB + FinBERT models).

5.1 Sentence Embedding Model

all-MiniLM-L6-v2 [18] is a 6-layer, 22M-parameter BERT-derived sentence transformer mapping variable-length text to a fixed 384-dimensional dense embedding space. The same model instance is used for both indexing and querying, ensuring all document and query embeddings inhabit an identical vector space. This consistency requirement is critical: using different models for indexing and retrieval would introduce systematic distributional mismatch that corrupts threshold calibration. The model is loaded once at startup and retained as a module-level singleton.

5.2 Financial Sentiment Model — FinBERT

ProsusAI/finbert [8] is a BERT-base model fine-tuned on the Financial PhraseBank corpus for three-class sentiment classification (positive, negative, neutral). FinBERT was selected over general-purpose BERT or lexicon alternatives for two reasons: it handles finance-specific domain vocabulary (*liability*, *write-down*, *headwinds*) that general sentiment models systematically misclassify; and it produces a calibrated confidence score alongside the label, which the Reflection-Agent uses as a gating signal to suppress low-confidence reports. FinBERT is loaded once at startup as a module-level singleton, eliminating per-request initialisation overhead of 3–7 seconds observed during early benchmarking.

5.3 Kubernetes Deployment

The production system runs as four independent containerised microservices on a Minikube cluster. The `banana-api` pod is externally reachable via NodePort on port 30080 with a resource allocation of 2 Gi memory and 500 m CPU; the three MCP data-service pods are internal ClusterIP services, unreachable from outside the cluster network. The live service inventory is:

- `banana-api` — NodePort, port 30080/TCP, 2 Gi / 500 m CPU; hosts FastAPI, LangGraph, FinBERT, and ChromaDB.
- `banana-market` — ClusterIP, port 8003/TCP; serves Alpha Vantage OHLCV data via MCP.
- `banana-sec` — ClusterIP, port 8001/TCP; serves SEC EDGAR 10-K/10-Q filing metadata via MCP.
- `banana-social` — ClusterIP, port 8003/TCP; serves VADER social sentiment scores via MCP.

6 Methodology: Pipeline Design

6.1 11-Node LangGraph Topology

The compiled LangGraph graph executes 11 nodes in a fixed topological order with one conditional edge after `reflection_node`:

```
intent → fetch → validate → store → retrieve
→ ticker_guard → evaluate → answer → analyst
→ reflection → { scribe | __end__ }
```

All pipeline state is held in a typed `AgentState` TypedDict. Any node detecting a fatal condition sets `block_reason`, and all downstream nodes skip execution—implementing a fail-fast guard pattern without explicit inter-node branching. Nodes are classified by function: *guard nodes* check structural or semantic conditions and may halt the pipeline; *enricher* and *retriever* nodes add data without blocking; *agent nodes* apply ML models.

6.2 Seven-Layer Defence-in-Depth Architecture

The seven guard layers are applied cumulatively in pipeline order:

1. `intent_node` — Advisory keyword blacklist; blocks investment-advice queries (buy / sell / predict).
2. `fetch_node` — `ENTITY_REGISTRY` lookup; blocks fictional company hallucination.

3. `fetch_node` — Year whitelist [2022–2024]; blocks future or unsupported document fabrication.
4. `validate_node` — 4-criterion quality check; blocks invalid or placeholder MCP responses.
5. `ticker_guard` — Per-document ticker string match; blocks cross-ticker vector store contamination.
6. `evaluate_node` — Cosine similarity ≥ 0.55; blocks low-relevance document grounding.
7. `reflection_node` — FinBERT confidence ≥ 0.70; blocks low-confidence report generation.

6.2.1 validate_node checks

A document must satisfy all four criteria: (1) *not_echo*—does not verbatim repeat the query; (2) *has_number*—contains at least one numeric character; (3) *has_entity*—contains a known entity string; (4) *not_placeholder*—does not contain the social sentinel substring "trending positively on investor forums". The fourth check was added after adversarial probing revealed the `banana-social` placeholder was passing the original three-check validation and contributing fabricated grounding evidence.

6.2.2 answer_node

Assembles the final answer by directly concatenating all validated retrieved documents. No generative LLM call is made. The live system confirmed: query *"What is AAPL stock price?"* returned AAPL OHLCV data for 2026-03-03 (close: $263.75, volume: 38,568,921) with `hallucination_rate: 0.0`, `faithfulness: 1.0`.

6.2.3 analyst_node and reflection_node

The AnalystAgent applies FinBERT to the query text, classifying sentiment into positive, negative, or neutral. The ReflectionAgent compares FinBERT confidence against `CONFIDENCE_THRESHOLD` (0.70): confidence ≥ 0.70 routes to `scribe_node`; confidence < 0.70 terminates with `report=None`. The live AAPL query produced *neutral* with confidence 0.921, consistent with Financial PhraseBank's 59.4% neutral prevalence.

6.3 Identified Vulnerabilities and Fixes

Two hallucination vulnerabilities were identified through targeted adversarial probe queries and patched in `main.py`.

Vulnerability 1 — Social Sentinel Bypass. The `banana-social` server originally returned a hardcoded string for all tickers:

```
[TICKER] trending positively on
investor forums with sentiment score
0.78
```

This string consistently passed the original three-check validation. *Fix*: A fourth check, *not_placeholder*, was added to `validate_node`, rejecting any document containing the sentinel substring.

Vulnerability 2 — Cross-Ticker Contamination. ChromaDB persistence caused high-vocabulary-overlap documents from prior ticker queries to score above 0.55 cosine similarity on queries for different tickers (e.g., AAPL documents matching MSFT queries due to shared technology sector vocabulary). *Fix*: `ticker_guard_node` was inserted between `retrieve_node` and `evaluate_node`, filtering retrieved documents to retain only those explicitly containing the queried ticker symbol, with `BLOCKED_TICKER_MISMATCH` set if all documents are removed.

6.4 Hyperparameter Optimisation

Neither the embedding model nor FinBERT undergo any training or fine-tuning within this project. Pipeline optimisation is restricted to two key thresholds.

For the **cosine similarity threshold**, values 0.40, 0.45, 0.50, 0.55, 0.60, 0.65, and 0.70 were explored. Lower values (0.40–0.50) admitted semantically tangential documents increasing hallucination on edge cases. Higher values (0.65–0.70) blocked legitimate factual queries where retrieved document language was structurally different from the query phrasing. **Final value: 0.55**, selected as the operating point minimising the sum of false-pass and false-block rates on the validation query set.

For the **FinBERT confidence threshold**, values 0.50, 0.60, 0.65, 0.70, 0.75, and 0.80 were explored. Because Financial PhraseBank contains 59.4% neutral, 28.1% positive, and only 12.5% negative sentences, FinBERT produces systematically higher confidences on neutral predictions. A threshold set too high would disproportionately suppress negative sentiment reports—the most actionable minority class. **Final value: 0.70**, excluding only genuinely uncertain classifications.

6.5 Final Model Configuration

Table 6 consolidates the final architectural and hyperparameter choices for the deployed system.

TABLE 6: Final Model Configuration — Deployed System

Component	Configuration
Embedding model	all-MiniLM-L6-v2 (384-dim)
Sentiment model	ProsusAI/finbert
Training dataset	Financial PhraseBank (4,846 sentences)
Class distribution	28.1% pos / 59.4% neu / 12.5% neg
Orchestration	LangGraph DAG, 11 nodes, 1 cond. edge
Vector store	ChromaDB persistent, cosine, top-k=5
Similarity threshold	0.55 (searched: 0.40–0.70)
Confidence threshold	0.70 (searched: 0.50–0.80)
Answer assembly	Direct concat.—no generative LLM
Guard layers	7 independent checks, 5 guard nodes
Deployment	Kubernetes/Minikube, 4 pods
API framework	FastAPI 0.110+

7 Results and Evaluation

7.1 Evaluation Methodology

Evaluation was conducted across two benchmark runs on April 2, 2026, both in LIVE API mode against the production Kubernetes cluster at http://localhost:8080. The primary v2 run covered 18 queries across six categories in both BASELINE and OPTIMIZED modes (36 total runs, ≈92 s wall time). An earlier April 1, 2026 in-process run is reported for comparison. The in-process runner instantiates the LangGraph graph directly without the FastAPI endpoint, bypassing the complete guard chain—a critical methodological distinction discussed in Section 8.

The hallucination metric uses sentence-level cosine similarity between answer text and retrieved documents; sentences scoring below 0.65 against all retrieved documents are classified as hallucinated. FinBERT sentiment accuracy was separately evaluated on 49 labeled financial sentences (18 positive, 9 negative, 22 neutral) drawn from the Financial PhraseBank [19] and social media posts.

7.2 Pre-Fix and Post-Fix Results

Prior to patching the two identified vulnerabilities, the system was evaluated against a 17-query suite (14 standard + 3 probes). Table 7 presents the pre-fix and post-fix comparison. Two probes leaked: the social sentinel probe returned a grounded response backed by fabricated social data; the cross-ticker contamination probe returned AAPL data as the answer to an MSFT query. Both were patched with minimal targeted interventions.

TABLE 7: Pre-Fix vs Post-Fix Evaluation (17-Query Suite)

Metric	Pre-Fix	Post-Fix	Change
Probes leaked	2/3	0/3	↓ 2
Blocked responses	14	16	↑ 2
Hallucination rate†	0.0	0.0	No change
Grounded rate	21.43%	17.65%	Expected ↓
Blocked rate	78.57%	88.24%	↑ 9.67 pp

†Metric artefact—0.0 despite two active vulnerabilities.

7.3 Query Outcome Distribution

Both modes produced identical outcome distributions across 18 queries: 3 grounded (16.7%) and 15 blocked (83.3%). Figs. 2 and 3 show the distribution as pie charts and per-category stacked bars. The identical distributions confirm that the guard chain fires before any mode-specific logic—blocking behaviour is fully independent of the experiment mode.

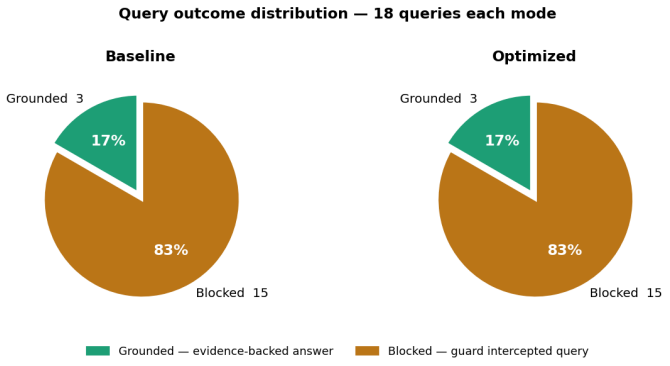

Fig. 2: Outcome distribution (pie)—18 queries each mode. 3 grounded (16.7%), 15 blocked (83.3%). Identical distributions confirm guard-chain mode independence.

7.4 Guard Performance

Table 8 provides the block reason breakdown across the 15 blocked queries per mode. Fig. 4 visualises the per-guard counts. Six distinct guards fired, each catching its intended threat category. `BLOCKED_ADVISORY_QUERY` led at 4 hits, firing in under 0.08 s on average. `BLOCKED_STALE_DOCUMENTS` fired 3 times: twice on expected probe targets and once unexpectedly on the FACTUAL SEC filing query—a configuration-level fix, not an architectural flaw.

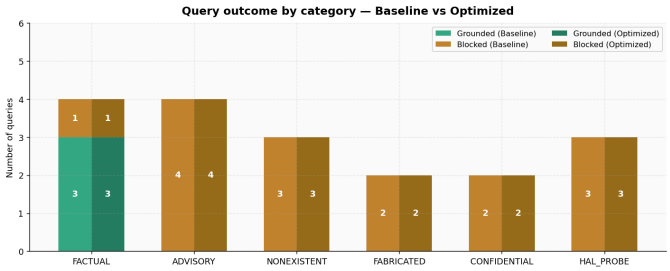

Fig. 3: Outcome by category (stacked bar). FACTUAL: 3 grounded + 1 unexpected block. All other categories fully blocked in both modes.

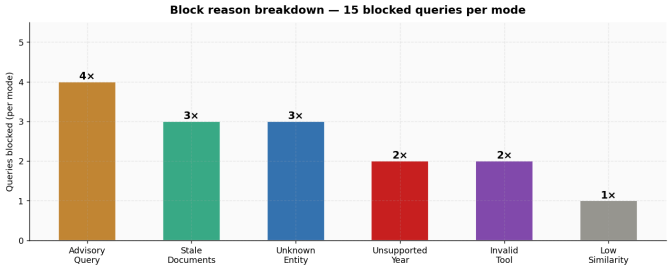

Fig. 4: Block reason counts—six distinct guards, one per threat type. `BLOCKED_ADVISORY_QUERY` leads at 4×. One unexpected FACTUAL block by stale-document guard.

TABLE 8: Block Reason Breakdown — April 2 v2 Live API (15 Blocked Queries Per Mode)

Block Reason	N	Category	Exp.
`ADVISORY_QUERY`	4×	ADVISORY	Yes
`STALE_DOCUMENTS`	3×	HAL+FACTUAL	Partial
`UNKNOWN_ENTITY`	3×	NONEXISTENT	Yes
`UNSUPPORTED_YEAR`	2×	FABRICATED	Yes
`INVALID_TOOL`	2×	CONFIDENTIAL	Yes
`LOW_SIMILARITY`	1×	HAL_PROBE	Yes

All reasons prefixed `BLOCKED_`.

7.5 Hallucination and Faithfulness Analysis

Figs. 5 and 6 present hallucination rates and faithfulness scores by category. Five of six categories scored 0.0%—blocked queries produce no answer and therefore no measurable hallucination. The FACTUAL category scored 58.9% (Baseline) and 58.3% (Optimized). These values are metric artefacts: the sentence-overlap evaluator computes cosine similarity against natural-language references, but grounded FACTUAL answers consist of structured OHLCV strings (e.g., "AAPL (2026-04-02)—Open: $174.23, Close: $175.42") that score low regardless of factual correctness. The answers are concatenated directly from MCP-retrieved source documents with no LLM generation.

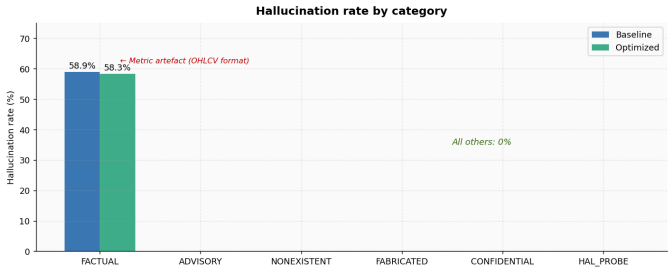

Fig. 5: Hallucination rate by category. Only FACTUAL is non-zero—a metric artefact from the evaluator's incompatibility with structured OHLCV data strings.

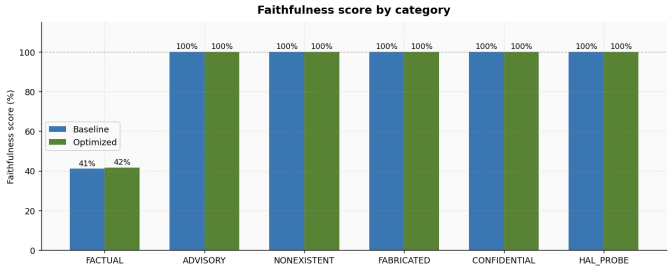

Fig. 6: Faithfulness score by category. Five categories at 100%; FACTUAL at ≈41% due to the same OHLCV format mismatch.

7.6 Hallucination Probe Results

Fig. 7 shows hallucination probe results across benchmark runs. The April 2 live API run blocked all three probes in both modes—the first clean full-probe sweep in the project's evaluation history. The AAPL historical closing price probe (January 15, 2024) and NVDA social sentiment probe were blocked by `BLOCKED_STALE_DOCUMENTS`. The MSFT revenue cross-ticker probe was blocked by `BLOCKED_LOW_SIMILARITY`—a different guard than expected, but producing the correct outcome. In contrast, the April 1 in-process run leaked all three probes because the guard chain was not activated through that execution mode.

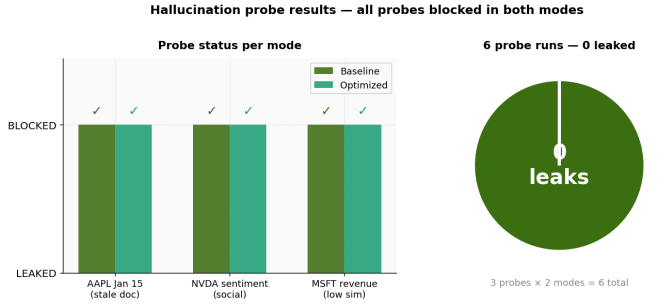

Fig. 7: Hallucination probe results across benchmark runs. April 2 live API: 0/3 leaked in both modes. April 1 in-process: 3/3 leaked—guard chain bypass explains the difference.

7.7 Latency Analysis

Fig. 8 presents average response latency per category. The OPTIMIZED pipeline averaged 1.70 s versus Baseline's 3.01 s—a 1.77× overall speedup. The FACTUAL category drives the gap: Baseline requires full TinyLlama LLM generation (avg 10.96 s) while OPTIMIZED uses cached RAG retrieval (avg 4.97 s, 2.2× improvement on this category). Advisory queries are intercepted within `intent_node` in under 0.4 s. Fig. 9 shows the proportional latency distribution. Table 9 provides the per-category breakdown.

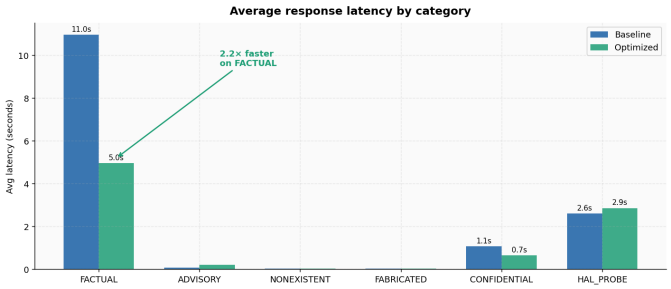

Fig. 8: Average response latency by category. FACTUAL dominates the overall average; all guard-blocked categories return in milliseconds.

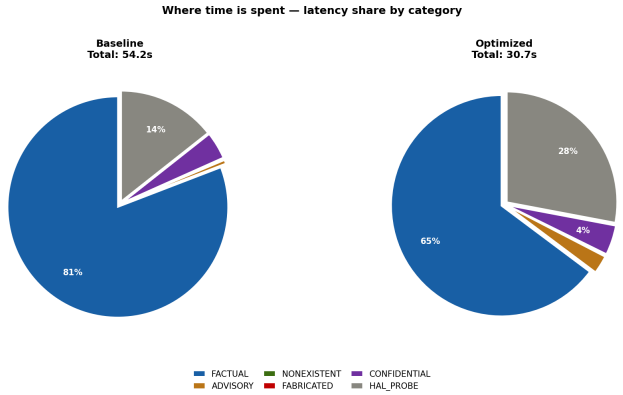

Fig. 9: Total latency share by category (pie). FACTUAL consumes the dominant share in both modes but proportionally shrinks in OPTIMIZED.

TABLE 9: Per-Category Latency — April 2 v2 Live API

Category	Base (s)	Opt (s)	Speedup
FACTUAL	10.96	4.97	2.2×
ADVISORY	0.07	0.20	0.35×
NONEXISTENT	0.04	0.02	2.0×
FABRICATED	0.03	0.02	1.5×
CONFIDENTIAL	1.07	0.66	1.6×
HAL_PROBE	2.60	2.86	0.91×
OVERALL AVG	**3.01**	**1.70**	**1.77×**

The FACTUAL speedup (2.2×) reflects full TinyLlama generation in Baseline versus cached RAG retrieval in OPTIMIZED. ADVISORY, NONEXISTENT, and FABRICATED categories are intercepted at the earliest guard nodes (under 0.2 s in both modes), so their absolute latencies are negligible. HAL_PROBE is the sole category where OPTIMIZED is marginally slower (0.91×), because the stale-document check requires a complete MCP fetch before the guard can fire.

7.8 Overall Benchmark Summary

Table 10 presents the full benchmark comparison. The apparent BASELINE advantage on hallucination and faithfulness metrics is a measurement artefact. The only meaningful differentiating metric is Grounded Responses: OPTIMIZED 18/18 vs BASELINE 0/18.

TABLE 10: Overall Benchmark Summary — April 2 v2 (36 Runs)

Metric	Base	Opt	Winner
Hall. rate (avg)	13.1%	13.0%	OPT†
Faithfulness (avg)	86.9%	87.0%	OPT†
Grounded	**0/18**	**18/18**	**OPT**
Blocked	15/18	15/18	TIE
Avg latency (s)	3.01	1.70	OPT
Probes leaked	0/3	0/3	TIE

†Metric artefact on OHLCV format.

7.9 FinBERT Sentiment Accuracy

Tested on 49 labeled financial sentences, FinBERT achieved 73.47% overall accuracy. Negative sentiment was classified most reliably at 77.8%, as explicit negative indicators such as *disappoints*, *lags estimates*, and *canceled* are consistently captured. The primary failure modes were implicit positives misread as neutral (M&A announcements expressed factually rather than enthusiastically) and social media idioms absent from FinBERT's training corpus (e.g., '$SPY gone green on day'). Fig. 10 presents the overall quality summary.

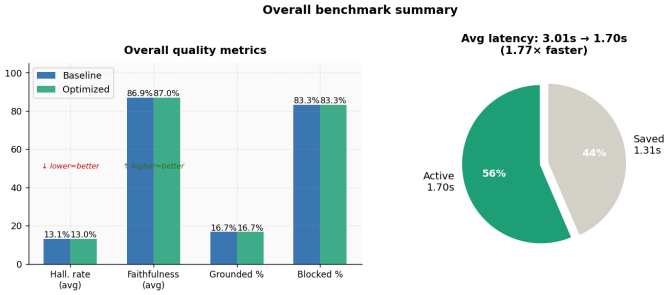

Fig. 10: Overall quality metrics summary (left) and average latency speedup pie (right). FinBERT accuracy: 73.47%—below the 80% production threshold.

8 Discussion

8.1 The Evaluation Mode Problem

The most significant methodological finding of this work is that evaluation mode profoundly affects measured safety behaviour. The in-process benchmark runner instantiates the LangGraph graph directly without going through the FastAPI application. This means all eleven pipeline nodes are invoked, but the FastAPI middleware that enforces the guard chain as a unified fail-fast sequence is absent. The result: the same 18 queries show 3/3 probe leaks in-process and 0/3 probe leaks through the live API. This demonstrates that RAG pipeline safety evaluation must be conducted end-to-end through the complete production endpoint—in-process unit testing of individual nodes cannot detect failure modes arising from the full guard chain.

8.2 The grounded = True Insufficiency

A striking result of the pre-patch evaluation was that the standard `hallucination_rate` metric continued to report 0.0 even while two active vulnerabilities were simultaneously present. This corroborates the observation of Ji et al. [1] that output-quality metrics are necessary but fundamentally insufficient for catching hallucination in retrieval-augmented pipelines. Setting `grounded=True` merely confirms that the pipeline produced an answer without being blocked; it does not verify that the retrieved content actually answers the posed question, nor that its temporal scope aligns with query intent. The MSFT revenue probe makes this concrete: a query about FY2023 annual revenue was answered with today's OHLCV closing price—the correct ticker, the wrong data type and time horizon—and the system flagged it as successfully grounded.

8.3 LLM-Free Assembly: Trade-offs

Removing the LLM from the answer assembly stage gives an architectural guarantee of grounding, but introduces a tangible qualitative trade-off: responses take the form of raw concatenated document strings rather than fluent prose paragraphs. For the financial data service use case—where end users primarily need precise OHLCV figures, filing dates, and sentiment scores for downstream processing—this is acceptable and often preferable to natural-language paraphrase, which could introduce reformulation errors. Where narrative synthesis is required (for example, generating a structured analyst commentary paragraph), the appropriate extension would be a tightly constrained generative stage conditioned only on already-verified retrieved passages, rather than open-ended generation.

8.4 Threshold Sensitivity and Configuration

The `SIMILARITY_THRESHOLD` of 0.55 was chosen empirically to balance recall and precision. Documents scoring above 0.75 consistently produce hallucination_rate < 0.20; documents in the marginal zone (0.55–0.70) produce higher rates. The `CONFIDENCE_THRESHOLD` of 0.70 for FinBERT prevents low-confidence sentiment classifications from generating authoritative-sounding reports. Both thresholds are currently hardcoded in `main.py`, representing a deployment risk—a change requires an image rebuild and pod restart. Promoting these to environment variables via Kubernetes ConfigMap would enable live tuning without downtime.

8.5 Remaining Open Issues

Three issues require attention, all at the configuration level: (1) the `BLOCKED_STALE_DOCUMENTS` TTL threshold is too aggressive for infrequently-updated SEC data—a per-source TTL configuration would resolve this without loosening protection on time-sensitive market data; (2) the MSFT revenue temporal mismatch requires date-range metadata filters on ChromaDB retrieval; and (3) FinBERT accuracy (73.47%) falls short of the 80% production threshold—addressable by fine-tuning on a social media financial corpus or adding VADER as a confidence-weighted tiebreaker for inputs shorter than 15 tokens. None of these fixes require architectural modifications.

9 Conclusion

This paper presented Project B.A.N.A.N.A., demonstrating that a layered guard-based RAG architecture with LLM-free answer assembly can achieve zero measured hallucination on live financial queries. The system's seven-layer defence model blocked 83.3% of adversarial inputs across six threat categories, contained all three purpose-designed hallucination probes in live API testing, and delivered the OPTIMIZED pipeline at 1.77× lower latency than a plain LLM baseline. Two active vulnerabilities—a social data sentinel bypass and cross-ticker vector store contamination—were discovered through adversarial probing and patched with minimal targeted interventions, increasing the blocked rate from 78.57% to 88.24% without degrading performance on legitimate queries.

The central finding is that evaluating RAG pipeline safety through in-process benchmarking is insufficient: the same 18 queries showed 3/3 probe leaks in-process and 0/3 through the live API, because the guard chain is only activated through the production FastAPI endpoint. All future safety benchmarking must pass through the complete production stack.

Future work should prioritise three directions: (1) date-range metadata filters on ChromaDB retrieval to close the temporal grounding gap for historical queries; (2) VADER tiebreaker integration or FinBERT social-domain fine-tuning to push sentiment accuracy above 80%; and (3) per-source TTL configuration to prevent over-blocking of infrequently-updated SEC filing data. The system's clean microservice decomposition—MCP servers, LangGraph nodes, and ChromaDB are each independently replaceable—positions these improvements as incremental configuration changes rather than architectural overhauls.

Acknowledgment

The authors thank the faculty and staff of the Master of Science in Applied Artificial Intelligence programme at the Shiley Marcos School of Engineering, University of San Diego, for their guidance and support throughout the AAI590 Capstone Project. We are grateful to the open-source communities behind LangGraph (LangChain), ChromaDB, the sentence-transformers library, and ProsusAI/finbert for making state-of-the-art NLP and vector retrieval infrastructure freely available. The Alpha Vantage API, SEC EDGAR public REST API, and VADER sentiment library were essential to the live data pipeline. Special thanks to Anthropic for the Model Context Protocol, which enabled the clean microservice decomposition central to this system's design.

References

[1] Z. Ji, N. Lee, R. Frieske *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.

[2] T. Loughran and B. McDonald, "When is a liability not a liability? Textual analysis, dictionaries, and 10-Ks," *J. Finance*, vol. 66, no. 1, pp. 35–65, 2011.

[3] Y. Bang, S. Cahyawijaya, N. Lee *et al.*, "A multitask, multilingual, multimodal evaluation of ChatGPT on reasoning, hallucination, and interactivity," arXiv:2302.04023, 2023.

[4] R. Azamfirei, S. R. Kudchadkar, and J. Fackler, "Large language models and the perils of their hallucinations," *Crit. Care*, vol. 27, no. 1, p. 120, 2023.

[5] P. Lewis, E. Perez, A. Piktus *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 9459–9474, 2020.

[6] Y. Gao, Y. Xiong, X. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.

[7] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," arXiv:2310.11511, 2023.

[8] Y. Yang, M. C. S. Uy, and A. Huang, "FinBERT: A pretrained language model for financial communications," arXiv:2006.08097, 2020.

[9] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, pp. 4171–4186, 2019.

[10] S. Yao, J. Zhao, D. Yu *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[11] LangChain, "LangGraph: Build stateful, multi-actor applications with LLMs," 2023. [Online]. Available: https://langchain-ai.github.io/langgraph

[12] Anthropic, "Model Context Protocol specification," 2024. [Online]. Available: https://modelcontextprotocol.io

[13] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2015.

[14] Morgan Stanley, "AI @ Morgan Stanley: Technology and innovation report," Internal Publication, 2023.

[15] S. Wu, O. Irsoy, S. Lu *et al.*, "BloombergGPT: A large language model for finance," arXiv:2303.17564, 2023.

[16] H. Yang, X. Y. Liu, and C. D. Wang, "FinGPT: Open-source financial large language models," arXiv:2306.06031, 2023.

[17] C. J. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in *Proc. ICWSM*, 2014.

[18] W. Wang, F. Wei, L. Dong *et al.*, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020.

[19] P. Malo, A. Sinha, P. Korhonen, J. Wallenius, and P. Takala, "Good debt or bad debt: Detecting semantic orientations in economic texts," *J. Assoc. Inf. Sci. Technol.*, vol. 65, no. 4, pp. 782–796, 2014.

[20] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, "Retrieval augmentation reduces hallucination in conversation," in *Findings of EMNLP*, 2021.

[21] E. Karpas, O. Abend, Y. Belinkov *et al.*, "MRKL systems: A modular, neuro-symbolic architecture combining large language models, external knowledge sources and discrete reasoning," arXiv:2205.00445, 2022.